Shi Cheng Huang

CSc 59866

Kurt Brown

Problem Understanding Paper

What problem are you investigating?

The problem that I am investigating is **reactive autoscaling for resource usage**, which means giving computer resources including **CPU** (central processing unit that processes instructions), **memory** (which stores active data), and I/O (input/output that reads and writes data) only after a threshold is crossed while working on a workload. As a reactive method, the system watches resource usage and only adds resources when CPU or memory crosses a threshold. This is inefficient when many different workloads must be managed, which leads to consequences such as slowing down the process and performance degradation within distributed computing systems.

To understand why this problem matters, consider a concrete example. A web service normally receives 100 requests per second. At a certain time, the traffic to the web service jumps to 500 requests per second within 30 seconds. The **Horizontal Pod Autoscaler** (HPA), a **Kubernetes** (a system for automating deployment and scaling of containerized applications) component that automatically adds or removes application instances when resource usage rises above or falls below a target threshold, checks metrics every 15–30 seconds. Thirty seconds later, it detects CPU above 80%, and 15 seconds after that, it starts scaling. Each new instance takes 60–120

seconds to start, after which new capacity is available. During the waiting period, existing instances are overloaded and lack resources. Response times increase, and some requests time out. Under normal conditions, reactive autoscaling works when resources are sufficient. It fails when resources are insufficient (under-provisioning), causing slowdowns and timeouts. It also wastes money when resources are excessive (over-provisioning), even though the service itself continues to function. Therefore, the right number of resources is required.

The boundaries of the problem cover only CPU and memory autoscaling in a single Kubernetes cluster. Although the problem statement includes I/O, this investigation excludes I/O prediction because I/O metrics are less standardized across Kubernetes workloads and would add scope beyond a single-student project. It also excludes multi-cluster scaling and GPU scheduling. The focus of the investigation is on bursty workloads such as spikes, where demand changes faster than the autoscaler can respond.

This problem matters because it costs money and hurts users' experience when workloads receive the wrong amount of resources. The problem occurs under conditions where resource demand changes so quickly that the system is unable to react, especially with unpredictable workloads. When demand for a workload is steady, the reactive method can function properly, but this is not always the case when demand is bursty.

Who or what is affected by it?

Cloud operators and platform engineers who run applications on distributed systems like Kubernetes (which automates the scaling and management of containerized applications) experience this problem. They manage large numbers of workloads that each require CPU,

memory, and I/O resources. They want to give each workload the right amount of resources — too little leads to slowdowns and crashes, while too much leads to wasted cost and capacity. End users of the applications are also affected by slower responses and performance degradation. The organization might permanently over-provision to prevent recurrence of the waiting period for resource allocation, which increases costs.

The consequences fall into three categories. First, missing information occurs during the lag window, when the system has no capacity for new requests, so some requests may be dropped. Second, repeated work occurs when engineers must manually intervene, and the same incident can recur because the original problem has not been fixed. Third, incorrect decisions occur when the organization chooses over-provisioning to be safe, which wastes money permanently. All three consequences trace back to the same root cause: the lag period between demand change and resource allocation.

What evidence do you currently have that it is a real problem?

I have not conducted any direct measurements. The evidence below focuses on CPU and memory usage because these are the resources most cited in literature. I/O evidence is not included because I/O metrics are less standardized across Kubernetes workloads, which is the same reason I/O is excluded from the investigation scope.

Reactive autoscaling is structurally lagging, with a three-to-five-minute timing period spent waiting to scale resources after the threshold is breached. During this window, it leads to tail-latency degradation (the slowest performance) and service-level objective (SLO) violations, where SLO is a target for service performance and a violation occurs when performance fails to

meet that target within a given time window. **[Documented]** This claim is documented by the thesis of Tran, Duy Anh as (https://trepo.tuni.fi/handle/10024/238698#1).

Conventional autoscaling, which is a reactive method, scales workloads based on the resource utilization of CPU and memory, which can lead to a failure to anticipate the burst demand of these workloads. This means that with the reactive method, the resources used for certain workloads cannot handle the burst demand for immediate changes in resources to achieve better performance. **[Documented]** This claim is documented by the abstract by Karthik Chakravarthy Cheekuri (https://www.jisem-journal.com/index.php/journal/article/view/13473/6334).

With sudden load spikes, Horizontal Pod Autoscaler will trigger the threshold and wait for the new resource, which can cause cold starts (the delay before a new instance is ready to serve requests), temporary under-provisioning, and frequent SLO violations. **[Documented]** This claim is documented by the poster abstract by Hyunsu Jeong and others (https://www.sc-asia.org/2026/data/poster/post240.pdf#1#1).

[Hypothesized] A Random Forest model trained on Alibaba trace will provide sufficient accuracy for practical autoscaling, this remains to be tested.

What is missing from this evidence is direct measurement in the target environment. The documented abstracts establish that the lag exists and that it causes SLO violations. They do not establish how large the lag is in a specific cluster, or how many SLO violations occur under a specific workload. These measurements would require running the experiment, which is the subject of the investigation plan.

What solutions already exist?

Two existing proactive approaches address this problem: Kairos and AROF. Both try to predict future demand instead of reacting after a threshold is reached.

Kairos (https://operatorhub.io/operator/kairos-operator) addresses the problem by using AI models and time-series forecasting to predict future CPU and memory usage, then adjusts resource allocation before demand arrives. Its strengths are intelligent resource allocation, multiple safe scaling modes, and conflict avoidance with other Kubernetes autoscaling tools. Its limitations are AI dependency, which requires an external endpoint for AI usage, and forecast accuracy, which depends on historical data. Under bursty conditions, Kairos relies on historical patterns, so a sudden pattern change not represented in the training data may not be anticipated.

AROF (https://www.tandfonline.com/doi/full/10.1080/17445760.2025.2605532) addresses the problem by combining hybrid classification (clustering and tagging) and multi-horizon LSTM (Long Short-Term Memory, a type of neural network that learns patterns in sequences over time) forecasting with cost optimization. Its strength is multi-horizon forecasting, which produces predictions over multiple future time horizons. Its limitations are model complexity with multi-horizon forecasting, as well as uncertainty, especially for a long time into the future. Under bursty conditions, AROF may require retuning for different workload patterns.

Resource usage prediction for autoscaling is the proposed solution for my project. It predicts the amount of computer resources required for a workload based on historical data using AI or machine learning.

What is the most important thing you still do not know?

The most important thing that I still do not know is **whether using a prediction interval (a range of plausible values) instead of a point prediction (a single value) will reduce SLO violations enough to justify the added complexity**. This uncertainty matters because it directly affects the design. If prediction intervals reduce SLO violations, the added complexity is justified. However, if the reduction is not sufficient, a point prediction design is preferable.

Investigation plan: I will run both versions of the scaler on the same Alibaba 2021 microservice trace. Version A is the point prediction design that scales when the prediction exceeds the threshold, while Version B is the prediction interval design that scales when its upper bound exceeds the threshold. I will measure the SLO violation rate and the total resource cost for both versions.

How the result influences design: If Version B reduces SLO violations by more than 20% compared to Version A, at less than 10% additional resource cost, I will use a prediction interval in my design. However, if the improvement is smaller, I will use point prediction in my design to reduce complexity. This design decision depends on the measured result.

The data for this investigation is the Alibaba 2021 microservice trace, which contains CPU and memory usage for thousands of microservices. The measurements are the SLO violation rate and whether predicted demand exceeds provisioned capacity. The scenario is a replay against a simulated Kubernetes cluster. The result will determine which version of the design I use.

Individual Scope

What I will investigate: I will build a predictive autoscaling component for Kubernetes that forecasts future CPU and memory demand and scales resources before the threshold is reached.

What I will exclude: I will not build a production-ready system, modify the Kubernetes control plane, address GPU scheduling, train deep learning models, or consider I/O. I will use a Random Forest model (a machine learning algorithm that builds many decision trees).

The comparison I will make: I will compare point prediction against interval prediction on two metrics: SLO violation rate and additional resource cost. The comparison will determine which design I use.

Tools and data: Python (programming language), scikit-learn (machine learning library in Python), the Alibaba 2021 trace (dataset for testing), and a lightweight Kubernetes simulator such as KEMU or KWOK.

Possible solution direction: A predictive scaler that uses an upper confidence bound to provision resources conservatively, reducing SLO violations without excessive over-provisioning.

Why it is manageable for one student: The data is public, the model is simple, the simulator can run on a laptop, and the evaluation of SLO violations and over-provisioning cost is straightforward to compute and analyze. The scope is bounded to a single cluster and a single class of workloads. The comparison is between two design mechanisms, not an entire system and its alternatives, which keeps the experiment bounded. Therefore, the experiment is manageable for one student.